

HDOC 28

Name: Keshav Mittal SAP ID: 590032934

Batch: 16

1. Algorithms

Algorithm for Program 1: Column-Major Input, Single Loop Output

1. Start
2. Read rows and cols. Validate that both are > 0 .
3. Declare a 2D variable-length array `arr[rows][cols]`.
4. Loop `j` from 0 to `cols-1` (outer loop). Inside, loop `i` from 0 to `rows-1` (inner loop). Read `arr[i][j]`.
5. Calculate `total_elements = rows * cols`.
6. Loop `k` from 0 to `total_elements-1`.
7. Calculate row index as `k / cols` and column index as `k % cols`. Print `arr[k / cols][k % cols]`.
8. Stop

Algorithm for Program 2: 2D to 1D Array Copy

1. Start
2. Read rows and cols. Validate that both are > 0 .
3. Declare a 2D VLA `arr2D[rows][cols]` and a 1D VLA `arr1D[rows * cols]`.
4. Loop `i` from 0 to `rows-1` and `j` from 0 to `cols-1`. Read elements into `arr2D[i][j]`.
5. In the same loop (or a separate nested loop), copy the elements: `arr1D[i * cols + j] = arr2D[i][j]`.
6. Display `arr2D` in matrix format using nested loops.
7. Display `arr1D` using a single loop from 0 to $(rows * cols) - 1$.
8. Stop

2. Test-Case Table

TC ID	Scenario Description	Inputs	Expected Output
TC-01	Invalid dimensions validation	rows: 0, cols: -2	Error message: Dimensions must be positive. Loops until valid input.
TC-02	Program 1: Col-major input, single loop output	rows: 2, cols: 2. Input: 1, 3, 2, 4	Output in row-major: 1 2 3 4
TC-03	Program 2: 2D to 1D copy	rows: 2, cols: 3. Input: 1, 2, 3, 4, 5, 6	2D output: Matrix form. 1D output: 1 2 3 4 5 6

3. C Program Source Code (Program 1)

```
#include <stdio.h>

int main() {
    int rows, cols;
    int i, j, k, total_elements;

    // a) Read number of rows and columns with validation
    do {
        printf("Enter number of rows and columns (> 0): ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0) {
            printf("Error: Both rows and columns must be positive integers.\n");
        }
    } while (rows <= 0 || cols <= 0);

    // b) Declare a variable-length 2D integer array
    int arr[rows][cols];

    // c) Read elements column by column
    printf("\nEnter %d elements column by column:\n", rows * cols);
    for (j = 0; j < cols; j++) {
        for (i = 0; i < rows; i++) {
            printf("Element [%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    // d) Print elements in row-major order using a single loop
    total_elements = rows * cols;
    printf("\nElements in row-major order:\n");
    for (k = 0; k < total_elements; k++) {
        // Linear index mapping: row = k / cols, col = k % cols
        printf("%d ", arr[k / cols][k % cols]);
    }
    printf("\n");

    return 0;
}
```

4. C Program Source Code (Program 2)

```
#include <stdio.h>

int main() {
    int rows, cols;
    int i, j, k;

    // Read number of rows and columns with validation
    do {
        printf("Enter number of rows and columns (> 0): ");
        scanf("%d %d", &rows, &cols);
        if (rows <= 0 || cols <= 0) {
```

```

        printf("Error: Both rows and columns must be positive integers.\n");
    }
} while (rows <= 0 || cols <= 0);

// Declare a 2D VLA and a 1D VLA int
arr2D[rows][cols];
int arr1D[rows * cols];

// Read elements into the 2D array and copy to 1D array printf("\nEnter %d elements for the 2D
array:\n", rows * cols); for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) { printf("Element [%d][%d]: ", i, j);
        scanf("%d", &arr2D[i][j]);

        // Copy to 1D array in row-major order arr1D[i * cols + j]
        = arr2D[i][j];
    }
}

// Display the 2D array
printf("\n2D Array (Matrix format):\n"); for (i = 0; i < rows;
i++) {
    for (j = 0; j < cols; j++) { printf("%4d ", arr2D[i][j]);
    }
    printf("\n");
}

// Display the 1D array
printf("\n1D Array (Row-major copy):\n"); for (k = 0; k <
(rows * cols); k++) {
    printf("%d ", arr1D[k]);
}
printf("\n");

return 0;
}

```

5. Compilation and Execution

```
(kali@kali)-[~/liak]
$ cc hdoc_28_1.c

(kali@kali)-[~/liak]
$ ./a.out
Enter number of rows and columns (>0): 4
4

Enter 16 elements for the 2D array:
Element [0][0]: 2
Element [0][1]: 3
Element [0][2]: 4
Element [0][3]: 5
Element [1][0]: 2
Element [1][1]: 2
Element [1][2]: 4
Element [1][3]: 5
Element [2][0]: 5
Element [2][1]: 3
Element [2][2]: 5
Element [2][3]: 3
Element [3][0]: 5
Element [3][1]: 3
Element [3][2]: 2
Element [3][3]: 4

2D Array (Matrix format):
  2   3   4   5
  2   2   4   5
  5   3   5   3
  5   3   2   4

1D Array (Row-major copy):
2 3 4 5 2 2 4 5 5 3 5 3 5 3 2 4
```

1.